Code generation for memory monitoring

Nikolaï Kosmatov Guillaume Petiot Julien Signoles

February 26, 2013

Abstract

We present:
* framework for memory monitoring
* instrumentation of C programs
* ...

1 Introduction

Memory accesses from a pointer or an array are not safe in C, thus trying to access an element of an array out of valid range or an invalid pointer may cause a segmentation fault during execution.

C compilers such as Gcc do not provide detection mechanisms for this kind of errors. We propose an automatic code instrumentation, so that the generated code will perform memory monitoring and will be able to check memory properties during execution (runtime checking). We consider as readable and writable memory: global variables, dynamically allocated memory, and formal parameters and local variables of each function. We decided to use the term "block" to nominate these four cases.

E-Acsl is an executable subset of Acsl [1], a formal specification language for C using source code annotations to express properties. Acsl-annotated C programs can be dealt with Frama-C [3], a framework for modular analysis of C. Frama-C provides a plug-in generating executable C code from E-Acsl annotations, used by the implementation discussed in this paper.

2 Related work

* ask Pascal ? *

3 Stored information

Memory blocks (as defined above) held properties such as their base address and their size. E-Acsl provides five annotations to retrieve useful informa-

tion about blocks:

`\base_addr(p)` returns the base address of the block containing pointer p

`\block_length(p)` returns the size (in bytes) of the block containing p

`\offset(p)` returns the offset between p and `\base_addr(p)`

`\valid_read(s)` whether reading $*s$ is safe

`\valid(s)` whether reading and writing $*s$ is safe

`\initialized(s)` whether s has been initialized

Our contribution allowed the E-Acsl plug-in to generate executable C code from these five annotations, thus checking these five memory properties during execution.

For each block, we need to store the following information: the base address (address of the first element within the block), the size (in bytes), the validity status (whether reading or writing the block is safe) and the initialization status for each byte of the block. We call *block descriptor* the data structure storing this information.

To keep the memory space occupied by this structure low, whenever none or all of the bytes of a memory block have been initialized (the two most common cases), the initialization array is freed; an integer field counting initialized bytes is used instead.

4 Global architecture

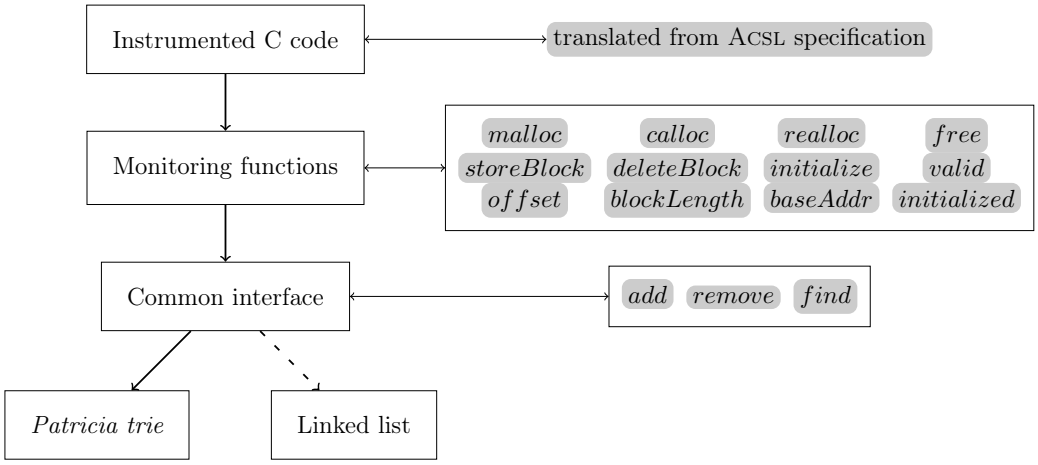

Figure 1: Global architecture

Interactions between the components of our solution are displayed by Figure 1. Informations about block descriptors are stored in an arbitrary data structure S (see Section 5). The E-Acsl plug-in performs an instrumentation to generate executable C code invoking monitoring functions (see Section 6) relying on the data structure S.

5 Concrete data structure used: *Patricia trie*

In this section we discuss the choice of the concrete data structure used to store the block descriptors.

We need a data structure with a good time and space complexity, indeed we may have to often add or remove a block descriptor in the structure. The structure has to be sorted: we want to access to a block descriptor by its base address, and also to its predecessor and successor. Thus, hash-tables will not fit. We also unconsidered the linked lists, due to the linear worst-case complexity. The (unbalanced) binary search trees provide a linear worst-case complexity too when the base address of inserted elements are monotonically increasing, and this may be quite common. Self-balanced binary search trees are dismissed because of the numerous add/remove operations implying numerous costly balancing operations.

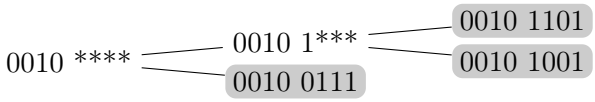

Figure 2: Example of Patricia trie

We choose to use the *Patricia trie* [4] structure, which is efficient even if the tree is unbalanced. The prefix used by a node (not a leaf) is the greatest common prefix (on 32 or 64 bits) of its two children. The block descriptors are held on leaves. Other nodes just do the routing from the root to a block descriptor.

For example on 8-bit addresses, Figure 2 shows a Patricia trie storing three block descriptors (identified by their addresses: `0010 0111`, `0010 1001` and `0010 1101`). The greatest common prefix of `0010 1001` and `0010 1101` is `0010 1` $***$ and the greatest common prefix of `0010 0111` and `0010 1` $***$ is `0010` $****$. The $*$ means that this bit is meaningless. Figure 3 shows that a new node (with a new prefix) is added when a block descriptor is inserted.

Figure 4 shows an example of deletion of a 8-bit block descriptor. Patricia tries being compact prefix trees, a node having an only child is deleted. So `0010 1` $***$ becomes useless and is replaced by its child `0010 1001`. Deleting useless nodes, or only storing useful ones for routing, keeps the tree exploration efficient. A 32-bit (respectively 64-bit) trie has a worst-case depth of 33 (respectively 65) nodes.

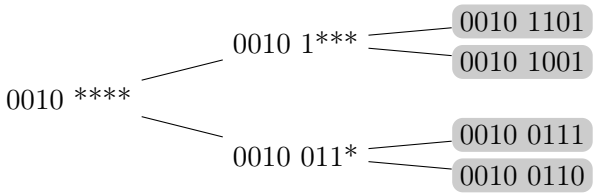

Figure 3: Patricia trie after adding `0010 0110` into the trie of Fig. 2

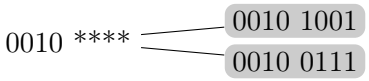

Figure 4: Patricia trie after deleting `0010 1101` from the trie of Fig. 2

5.1 Greatest common prefix computation

The greatest common prefix of A and B can be naively computed by: $X = \neg(A \oplus B)$ (where $\oplus$ is the XOR operator) to keep bits in common, then each bit of X on the right side of a 0 is set to 0. The obtained mask is then applied to A or B. For example, considering $A =$ `0110 0111` and $B =$ `0111 1111`, $X = \neg(A \oplus B) =$ `1110 0111`, setting each bit on the right side of a 0 to 0 we get `1110 0000`. We apply this mask to A and get the greatest common prefix of A and B: `011` $*$ $*$ $*$ $*$.

We optimized this algorithm by firstly computing all of the 65 or 33 different masks (from `0x0` to `0xf...f`) and storing them in an array. Then we use a dichotomic search to find the mask corresponding to the greatest common prefix: if A and B have a 32-bit common prefix, do they have a 48-bit common prefix ? Otherwise, do they have a 16-bit common prefix ? And so on. This search takes at most 6 (respectively 5) steps on 64-bit (respectively 32-bit) tries.

6 Monitoring functions

This section presents the functions used for adding/removing/retrieving in formations about block descriptors into our data structure (Patricia trie). These functions will be automatically inserted in the source code by the instrumentation performed by the E-Acsl plug-in.

6.1 Automatic allocation

Automatic allocation functions keep track of all non-dynamically allocated variables (but occupying memory space nevertheless), such as formal parameters, local variables or global variables.

```
1 { // begin of scope of v
2   T v;
3   storeBlock(v, sizeof(T));
4   ...
5   deleteBlock(v);
6 } // end of scope of v
```

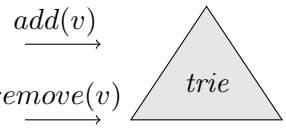

Figure 5: *storeBlock* and *deleteBlock*

6.2 Dynamic allocation

These functions have to be used instead of *malloc*, *realloc*, *calloc* and *free* from the standard library.

```
1   T* v;
2   v = malloc(sizeof(N));
3   ...
4   free(v);
```

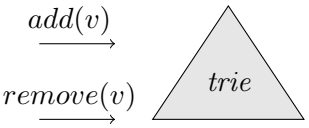

Figure 6: *malloc* and *free*

6.3 Initialization

These functions have to be used for each assignment, to update the initialization status of a block descriptor. $initialize(ptr, size)$ marks the $size$ bytes starting from ptr as initialized. $fullInit(ptr)$ marks all the bytes of ptr as initialized at once. It is designed to avoid multiple calls to *initialize* whenever it is possible and improves efficiency of the instrumented program.

```
1   v = (T)x;
2   initialize(v, sizeof(T));
```

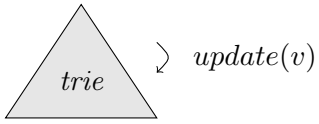

Figure 7: *initialize*

6.4 Interrogation

These functions are used to retrieve information about the block descriptors and match the E-ACSL annotations we are trying to support: `\valid`,

`\valid_read`, `\base_addr`, `\block_length`, `\offset` and `\initialized`.

valid(*ptr*, *size*) returns 1 if it is safe to read/write *size* bytes starting from *ptr*, 0 otherwise. *baseAddr*(*ptr*) returns the base address of the block containing *ptr* if such a block exists, NULL otherwise. *blockLength*(*ptr*) returns the size (in bytes) of the block containing *ptr* if such a block exists, 0 otherwise. $offset(ptr)$ returns the offset between *ptr* and the base address of the block containing *ptr* if such a block exists, -1 otherwise. *initialized*(*ptr*, *size*) returns 1 if the *size* first bytes starting from *ptr* are initialized, 0 otherwise.

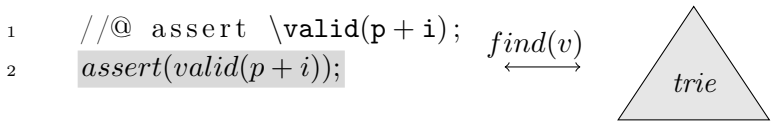

Figure 8: *valid* assertion checking

7 Experimental results

We used mutational testing to stress our library. The program under test is previously annotated (its specification is expressed in ACSL [1]. The program under test is supposed correct, we want to show that E-ACSL is able to spot the specification violations of "mutated programs" (that do not match the specification anymore).

We used a test generator granting path coverage [2] to generate test cases. We then use E-ACSL to instrument the program under test and the test cases, to generate calls to our monitoring framework. We the check that the instrumented original program (supposed correct) successfully pass each of those tests. We then generate mutated programs, obtained by modifying an element in the syntactic tree. The mutations used for these experiments are: numerical-arithmetic operator modification, pointer-arithmetic operator modification, comparison operator modification and logic (*land* and *lor*) operator modification. Then each mutant program is instrumented by E-ACSL.

Finally, each pair (*testcase*, *mutant*) is executed. The results are displayed in Fig. 9. We consider a mutant "killed" if there is at least one test case provoking a runtime error and this error has been found by E-ACSL.

	alarms	time	test cases	mutants	mutants killed
$fibonacci$	31	54s	38	20	20
$Bsearch$	28	63s	20	41	41
$Bsort$	37	273s	153	45	47
$Merge$	55	100s	19	58	58
$BubbleSort$	31	963s	873	44	44
$MultiplyMatrix$	44	146s	9	44	44

Figure 9: Experimental results

8 Conclusion

We implemented an efficient data structure to store the block descriptors and functions relying on this structure to perform memory monitoring. We also defined and implemented into the E-ACSL plug-in the instrumentation to perform on a C source code to monitor the memory. This allows the runtime checking of the following E-ACSL annotations: `\base_addr`, `\block_length`, `\offset`, `\valid`, `\valid_read` and `\initialized`.

References

[1] P. Baudin, P. Cuoq, J. C. Filliâtre, C. Marché, B. Monate, Y. Moy, and V. Prevosto. ACSL: ANSI/ISO C Specification Language preliminary design version 1.5. `http://frama-c.com/acsl.html`, 2010.

[2] B. Botella, M. Delahaye, S. Hong Tuan Ha, N. Kosmatov, P. Mouy, M. Roger, and N. Williams. Automating structural testing of C programs: Experience with PathCrawler. In *AST*, pages 70–78, 2009.

[3] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski. Frama-C: A program analysis perspective. In *Proc. of the 10th International Conference on Software Engineering and Formal Methods*, SEFM '2012, October 2012. To appear.

[4] W. Szpankowski. Patricia tries again revisited. *J. ACM*, 37(4):691–711, Oct. 1990.